

Dr. AMBEDKAR INSTITUTE OF TECHNOLOGY

(An Autonomous Institute, Affiliated to Visvesvaraya Technological University, Belagavi, Accredited by NAAC, with “A” Grade)

Near Jnana Bharathi Campus, Bangalore – 560056



DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

**Master of Technology
in
Computer Science and Engineering**

**“ADVANCES IN DATABASE MANAGEMENT SYSTEM
LABREPORT”**

Submitted by

SRIPADA RAO H 1DA25SCS18

For the academic year 2026 - 2027

Under the Guidance of

Dr. Shamshekhar S Patil

**AssociateProfessor
Dept. of CSE, Dr. AIT**



Visvesvaraya Technological University

Jnana Sangama, Belagavi, Karnataka 590018

2026 - 2027

Dr. AMBEDKAR INSTITUTE OF TECHNOLOGY

(An Autonomous Institute, affiliated to VTU, Belagavi, Accredited by NAAC with 'A' Grade)

Near Jnana Bharathi Campus, Bengaluru – 560056



CERTIFICATE

“ADBMS Laboratory”

This is to certify that the submitted document in the partial fulfillment of the requirement of the M.Tec 2nd semester ADBMS laboratory curriculum during the year 2025-26 is a result of Bonafede work carried out by-

SRIPADA RAO H

1DA25SCS18

Signature of the Incharge,

Dr. Shamshekhar S Patil
Associate Professor

Dr. AIT CSE

1. Internal Examiner _____

Dr.Nandhini

Professor & HOP,
CSE, Dr.AIT

TABLE OF CONTENTS

Program No.	Title	Page No.
Program 1	Library Management System	1
Program 2	College Database	9
Program 3	Employee Databases	20
Program 4	CRUD operations using an open-source NoSQL database (MongoDB).	22

PROGRAM 1

A. Consider the following schema for a Library Database:

BOOK (*Book_id*, Title, Publisher_Name, Pub_Year)

BOOK_AUTHORS (*Book_id*, Author_Name)

PUBLISHER (*Name*, Address, Phone)

BOOK_COPIES (*Book_id*, Branch_id, No-of_Copies)

BOOK_LENDING (*Book_id*, Branch_id, Card_No, Date_Out, Due_Date)

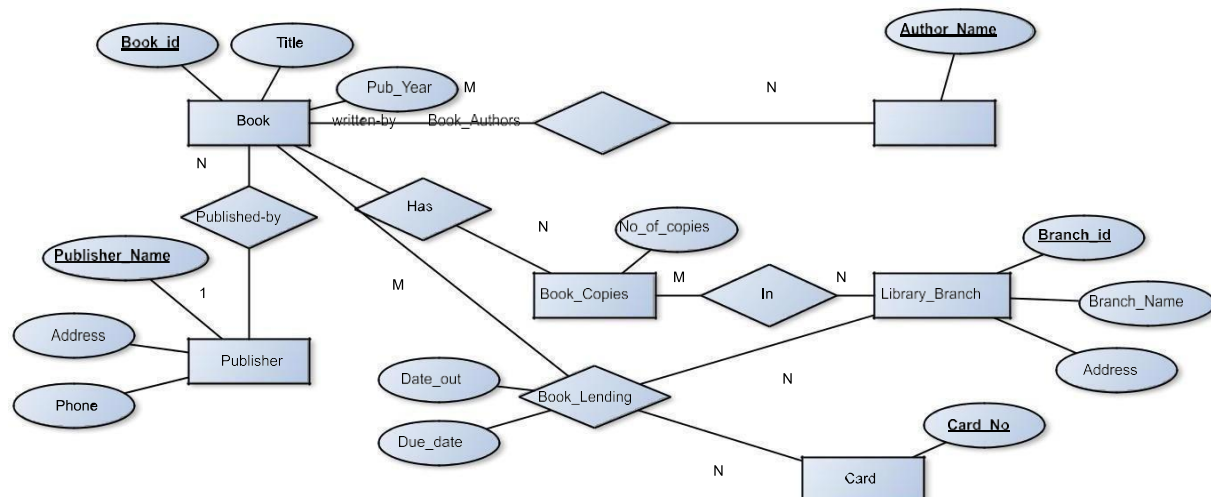
LIBRARY_BRANCH (*Branch_id*, Branch_Name, Address)

Write SQL queries to

1. Retrieve details of all books in the library – id, title, name of publisher, authors, number of copies in each branch, etc.
2. Get the particulars of borrowers who have borrowed more than 3 books, but from Jan 2017 to Jun 2017
3. Delete a book in BOOK table. Update the contents of other tables to reflect this data manipulation operation.
4. Partition the BOOK table based on year of publication. Demonstrate its working with a simple query.
5. Create a view of all books and its number of copies that are currently available in the Library.

Solution:

Entity-Relationship Diagram



Schema Diagram

Book

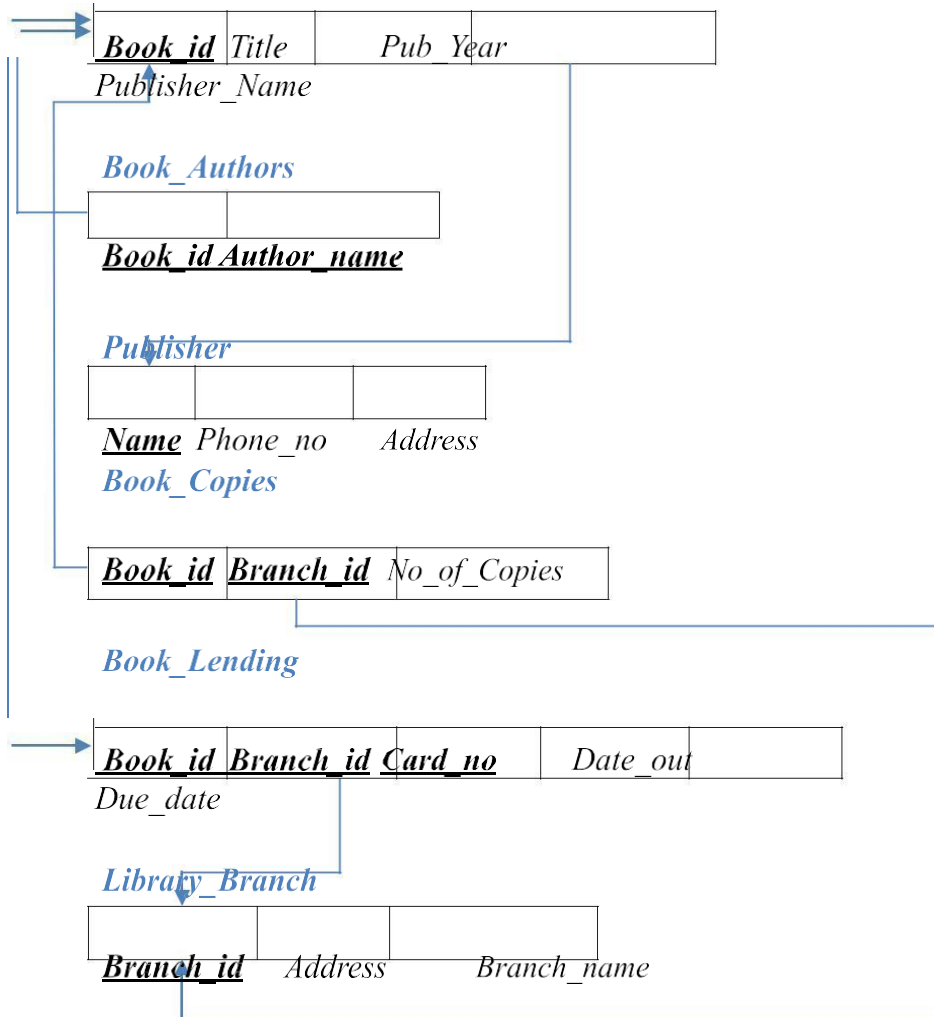


Table Creation

```
CREATE TABLE PUBLISHER  
(NAME VARCHAR2 (20) PRIMARY KEY,  
PHONE INTEGER,  
ADDRESS VARCHAR2 (20));
```

```
CREATE TABLE BOOK  
(BOOK_ID INTEGER PRIMARY KEY,  
TITLE VARCHAR2 (20),  
PUB_YEAR VARCHAR2 (20),  
PUBLISHER_NAME REFERENCES PUBLISHER (NAME) ON DELETE CASCADE);
```

```
CREATE TABLE BOOK_AUTHORS
(AUTHOR_NAME VARCHAR2 (20),
BOOK_ID REFERENCES BOOK (BOOK_ID) ON DELETE CASCADE,
PRIMARY KEY (BOOK_ID, AUTHOR_NAME));
```

```
CREATE TABLE LIBRARY_BRANCH
(BRANCH_ID INTEGER PRIMARY KEY,
BRANCH_NAME VARCHAR2 (50),
ADDRESS VARCHAR2 (50));
```

```
CREATE TABLE BOOK_COPIES
(NO_OF_COPIES INTEGER,
BOOK_ID REFERENCES BOOK (BOOK_ID) ON DELETE CASCADE,
BRANCH_ID REFERENCES LIBRARY_BRANCH (BRANCH_ID) ON DELETE CASCADE,
PRIMARY KEY (BOOK_ID, BRANCH_ID));
```

```
CREATE TABLE CARD
(CARD_NO INTEGER PRIMARY KEY);
```

```
CREATE TABLE BOOK_LENDING
(DATE_OUT DATE,
DUE_DATE DATE,
BOOK_ID REFERENCES BOOK (BOOK_ID) ON DELETE CASCADE,
BRANCH_ID REFERENCES LIBRARY_BRANCH (BRANCH_ID) ON DELETE CASCADE,
CARD_NO REFERENCES CARD (CARD_NO) ON DELETE CASCADE,
PRIMARY KEY (BOOK_ID, BRANCH_ID, CARD_NO));
```

Table Descriptions

DESC PUBLISHER;

```
SQL> desc publisher;
```

Name	Null?	Type
NAME	NOT NULL	VARCHAR2(20)
PHONE		NUMBER(38)
ADDRESS		VARCHAR2(20)

DESC BOOK;

```
SQL> DESC BOOK;
```

Name	Null?	Type
BOOK_ID	NOT NULL	NUMBER(38)
TITLE		VARCHAR2(20)
PUB_YEAR		VARCHAR2(20)
PUBLISHER_NAME		VARCHAR2(20)

DESC BOOK_AUTHORS;

```
SQL> DESC BOOK_AUTHORS;
Name                                     Null?      Type
-----
AUTHOR_NAME                             NOT NULL   VARCHAR2(20)
BOOK_ID                                  NOT NULL   NUMBER(38)
```

DESC LIBRARY_BRANCH;

```
SQL> DESC LIBRARY_BRANCH;
Name                                     Null?      Type
-----
BRANCH_ID                               NOT NULL   NUMBER(38)
BRANCH_NAME                             NULL       VARCHAR2(50)
ADDRESS                                 NULL       VARCHAR2(50)
```

DESC BOOK_COPIES;

```
SQL> DESC BOOK_COPIES;
Name                                     Null?      Type
-----
NO_OF_COPIES                             NULL       NUMBER(38)
BOOK_ID                                  NOT NULL   NUMBER(38)
BRANCH_ID                                NOT NULL   NUMBER(38)
```

DESC CARD;

```
SQL> DESC CARD;
Name                                     Null?      Type
-----
CARD_NO                                  NOT NULL   NUMBER(38)
```

DESC BOOK_LENDING;

```
SQL> desc book_lending;
Name
-----
DATE_OUT
DUE_DATE
BOOK_ID
BRANCH_ID
CARD_NO
```

Insertion of Values to Tables

```
INSERT INTO PUBLISHER VALUES ('MCGRAW-HILL', 9989076587, 'BANGALORE');
INSERT INTO PUBLISHER VALUES ('PEARSON', 9889076565, 'NEWDELHI');
INSERT INTO PUBLISHER VALUES ('RANDOM HOUSE', 7455679345, 'HYDRABAD');
INSERT INTO PUBLISHER VALUES ('HACHETTE LIVRE', 8970862340, 'CHENAI');
INSERT INTO PUBLISHER VALUES ('GRUPO PLANETA', 7756120238, 'BANGALORE');
```

```

INSERT INTO BOOK VALUES (1,'DBMS','JAN-2017', 'MCGRAW-HILL');
INSERT INTO BOOK VALUES (2,'ADBMS','JUN-2016', 'MCGRAW-HILL');
INSERT INTO BOOK VALUES (3,'CN','SEP-2016', 'PEARSON');
INSERT INTO BOOK VALUES (4,'CG','SEP-2015', 'GRUPO PLANETA');
INSERT INTO BOOK VALUES (5,'OS','MAY-2016', 'PEARSON');

```

```

INSERT INTO BOOK_AUTHORS VALUES ('NAVATHE', 1); INSERT
INTO BOOK_AUTHORS VALUES ('NAVATHE', 2); INSERT INTO
BOOK_AUTHORS VALUES ('TANENBAUM', 3); INSERT INTO
BOOK_AUTHORS VALUES ('EDWARD ANGEL', 4); INSERT INTO
BOOK_AUTHORS VALUES ('GALVIN', 5);

```

```

INSERT INTO LIBRARY_BRANCH VALUES (10,'RR NAGAR','BANGALORE');
INSERT INTO LIBRARY_BRANCH VALUES (11,'EWIT','BANGALORE');
INSERT INTO LIBRARY_BRANCH VALUES (12,'RAJAJI NAGAR', 'BANGALORE');
INSERT INTO LIBRARY_BRANCH VALUES (13,'NITTE','MANGALORE');
INSERT INTO LIBRARY_BRANCH VALUES (14,'MANIPAL','UDUPI');

```

```

INSERT INTO BOOK_COPIES VALUES (10, 1, 10);
INSERT INTO BOOK_COPIES VALUES (5, 1, 11);
INSERT INTO BOOK_COPIES VALUES (2, 2, 12);
INSERT INTO BOOK_COPIES VALUES (5, 2, 13);
INSERT INTO BOOK_COPIES VALUES (7, 3, 14);
INSERT INTO BOOK_COPIES VALUES (1, 5, 10);
INSERT INTO BOOK_COPIES VALUES (3, 4, 11);

```

```

INSERT INTO CARD VALUES (100);
INSERT INTO CARD VALUES (101);
INSERT INTO CARD VALUES (102);
INSERT INTO CARD VALUES (103);
INSERT INTO CARD VALUES (104);

```

```

INSERT INTO BOOK_LENDING VALUES ('01-JAN-17','01-JUN-17', 1, 10, 101);
INSERT INTO BOOK_LENDING VALUES ('11-JAN-17','11-MAR-17', 3, 14, 101);
INSERT INTO BOOK_LENDING VALUES ('21-FEB-17','21-APR-17', 2, 13, 101);
INSERT INTO BOOK_LENDING VALUES ('15-MAR-17','15-JUL-17', 4, 11, 101);
INSERT INTO BOOK_LENDING VALUES ('12-APR-17','12-MAY-17', 1, 11, 104);
SELECT * FROM PUBLISHER;

```

```
SQL> select * from publisher;
```

NAME	PHONE	ADDRESS
MCGRAW-HILL	9989076587	BANGALORE
PEARSON	9889076565	NEWDELHI
RANDOM HOUSE	7455679345	HYDRABAD
HACHETTE LIVRE	8970862340	CHENAI
GRUPO PLANETA	7756120238	BANGALORE

SELECT * FROM BOOK;

SQL> SELECT * FROM BOOK;

BOOK_ID	TITLE	PUB_YEAR	PUBLISHER_NAME
1	DBMS	JAN-2017	MCGRAW-HILL
2	ADBMS	JUN-2016	MCGRAW-HILL
3	CN	SEP-2016	PEARSON
4	CG	SEP-2015	GRUPO PLANETA
5	OS	MAY-2016	PEARSON

SELECT * FROM BOOK_AUTHORS;

SQL> SELECT * FROM BOOK_AUTHORS;

AUTHOR_NAME	BOOK_ID
NAVATHE	1
NAVATHE	2
TANENBAUM	3
EDWARD ANGEL	4
GALVIN	5

SELECT * FROM LIBRARY_BRANCH;

SQL> SELECT * FROM LIBRARY_BRANCH;

BRANCH_ID	BRANCH_NAME	ADDRESS
10	RR NAGAR	BANGALORE
11	RNSIT	BANGALORE
12	RAJAJI NAGAR	BANGALORE
13	NITTE	MANGALORE
14	MANIPAL	UDUPI

SELECT * FROM BOOK_COPIES;

SQL> SELECT * FROM BOOK_COPIES;

NO_OF_COPIES	BOOK_ID	BRANCH_ID
10	1	10
5	1	11
2	2	12
5	2	13
7	3	14
1	5	10
3	4	11

SELECT * FROM CARD;

SQL> SELECT * FROM CARD;

CARD_NO
100
101
102
103
104

```
SELECT * FROM BOOK_LENDING;
```

```
SQL> select * from book_lending;
```

DATE_OUT	DUE_DATE	BOOK_ID	BRANCH_ID	CARD_NO
01-JAN-17	01-JUN-17	1	10	101
11-JAN-17	11-MAR-17	3	14	101
21-FEB-17	21-APR-17	2	13	101
15-MAR-17	15-JUL-17	4	11	101
12-APR-17	12-MAY-17	1	11	104

Queries:

1. Retrieve details of all books in the library – id, title, name of publisher, authors, number of copies in each branch, etc.

```
SELECT B.BOOK_ID, B.TITLE, B.PUBLISHER_NAME, A.AUTHOR_NAME,
C.NO_OF_COPIES, L.BRANCH_ID
FROM BOOK B, BOOK_AUTHORS A, BOOK_COPIES C, LIBRARY_BRANCH L
WHERE B.BOOK_ID=A.BOOK_ID
AND B.BOOK_ID=C.BOOK_ID
AND L.BRANCH_ID=C.BRANCH_ID;
```

2. Get the particulars of borrowers who have borrowed more than 3 books, but from Jan 2017 to Jun 2017.

```
SELECT CARD_NO FROM
BOOK_LENDING
WHERE DATE_OUT BETWEEN '01-JAN-2017' AND '01-JUL-2017'
GROUP BY CARD_NO
HAVING COUNT (*)>3;
CARD_NO
-----
101
```

3. Delete a book in BOOK table. Update the contents of other tables to reflect this data manipulation operation.

```
SQL> DELETE FROM BOOK
2 WHERE BOOK_ID=3;
```

```
1 row deleted.
```

```
SQL> SELECT * FROM BOOK;
```

BOOK_ID	TITLE	PUB_YEAR	PUBLISHER_NAME
1	DBMS	JAN-2017	MCGRAW-HILL
2	ADBMS	JUN-2016	MCGRAW-HILL
4	CG	SEP-2015	GRUPO PLANETA
5	OS	MAY-2016	PEARSON

4. Partition the BOOK table based on year of publication. Demonstrate its working with a simple query.

```
CREATE VIEW V_PUBLICATION AS
SELECT PUB_YEAR
FROM BOOK;
```

PUB_YEAR
JAN-2017
JUN-2016
SEP-2016
SEP-2015
MAY-2016

5. Create a view of all books and its number of copies that are currently available in the Library.

```
CREATE VIEW V_BOOKS AS
SELECT B.BOOK_ID, B.TITLE, C.NO_OF_COPIES
FROM BOOK B, BOOK_COPIES C, LIBRARY_BRANCH L
WHERE B.BOOK_ID=C.BOOK_ID
AND C.BRANCH_ID=L.BRANCH_ID;
```

BOOK_ID	TITLE	NO_OF_COPIES
1	DBMS	10
1	DBMS	5
2	ADBMS	2
2	ADBMS	5
3	CN	7
5	OS	1
4	CG	3

PROGRAM 2

Consider the schema for College Database:

STUDENT (USN, SName, Address, Phone, Gender)

SEMSEC (SSID, Sem, Sec)

CLASS (USN, SSID)

SUBJECT (Subcode, Title, Sem, Credits)

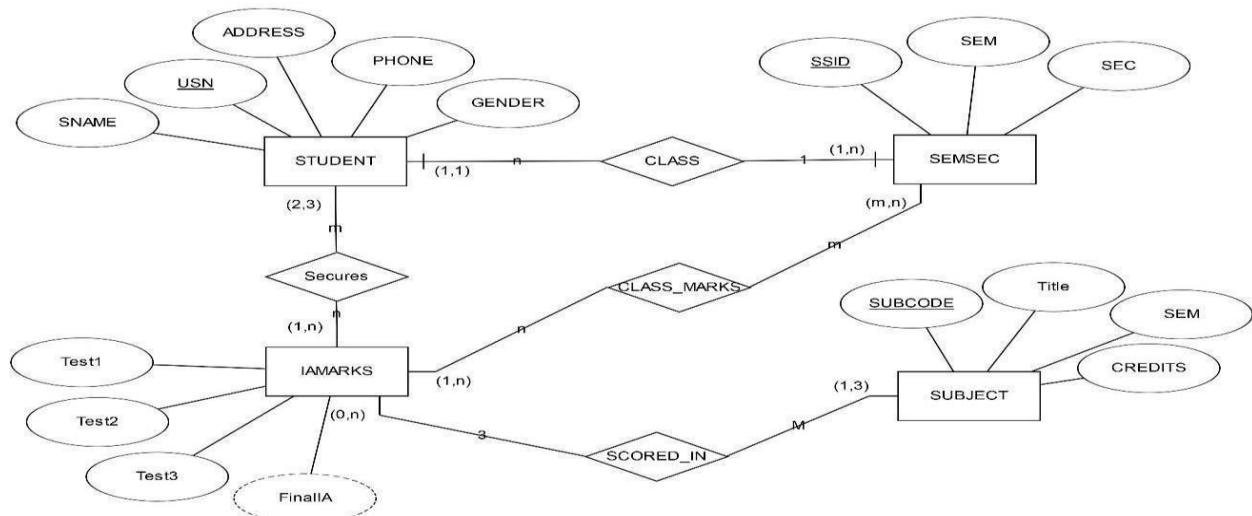
IAMARKS (USN, Subcode, SSID, Test1, Test2, Test3, FinalIA)

Write SQL queries to

1. List all the student details studying in fourth semester 'C' section.
2. Compute the total number of male and female students in each semester and in each section.
3. Create a view of Test1 marks of student USN '1BI15CS101' in all subjects.
4. Calculate the FinalIA (average of best two test marks) and update the corresponding table for all students.
5. Categorize students based on the following criterion:
If FinalIA = 17 to 20 then CAT = 'Outstanding'
If FinalIA = 12 to 16 then CAT = 'Average'
If FinalIA < 12 then CAT = 'Weak'
Give these details only for 8th semester A, B, and C section students.

Solution:

Entity - Relationship Diagram



Schema Diagram

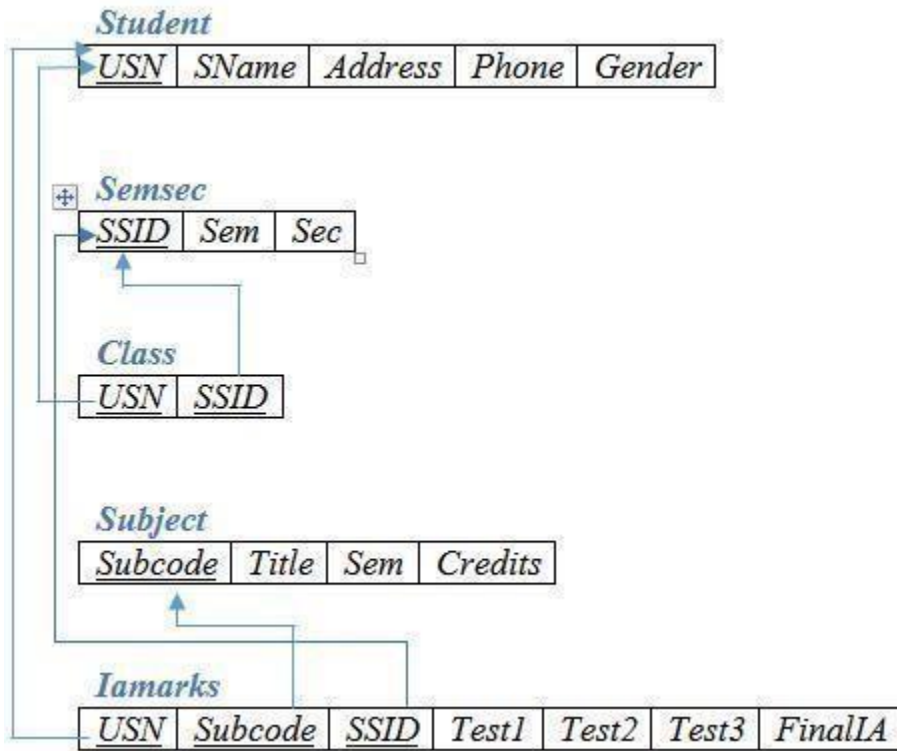


Table Creation

```
CREATE TABLE STUDENT (  
  USN VARCHAR (10) PRIMARY KEY,  
  SNAME VARCHAR (25),  
  ADDRESS VARCHAR (25),  
  PHONE NUMBER (10),  
  GENDER CHAR (1));
```

```
CREATE TABLE SEMSEC (  
  SSID VARCHAR (5) PRIMARY KEY,  
  SEM NUMBER (2),  
  SEC CHAR (1));
```

```
CREATE TABLE CLASS (  
  USN VARCHAR (10),  
  SSID VARCHAR (5),  
  PRIMARY KEY (USN, SSID),  
  FOREIGN KEY (USN) REFERENCES STUDENT (USN),  
  FOREIGN KEY (SSID) REFERENCES SEMSEC (SSID));
```

```
CREATE TABLE SUBJECT (  
  SUBCODE VARCHAR (8),  
  TITLE VARCHAR (20),
```

```
SEM NUMBER (2),
CREDITS NUMBER (2),
PRIMARY KEY (SUBCODE));
```

```
CREATE TABLE IAMARKS (
USN VARCHAR (10),
SUBCODE VARCHAR (8),
SSID VARCHAR (5),
TEST1 NUMBER (2),
TEST2 NUMBER (2),
TEST3 NUMBER (2),
FINALIA NUMBER (2),
PRIMARY KEY (USN, SUBCODE, SSID),
FOREIGN KEY (USN) REFERENCES STUDENT (USN),
FOREIGN KEY (SUBCODE) REFERENCES SUBJECT (SUBCODE), FOREIGN KEY
(SSID) REFERENCES SEMSEC (SSID));
```

Table Descriptions

```
DESC STUDENT;
```

```
Name
```

```
-----
USN
SNAME
ADDRESS
PHONE
GENDER
```

```
DESC SEMSEC;
```

```
DESC SEMSEC;
```

```
SQL> DESC SEMSEC;
```

```
Name
```

```
-----
SSID
SEM
SEC
```

```
DESC CLASS;
```

```
SQL> DESC CLASS;
```

```
Name
```

```
-----
USN
SSID
```

DESC SUBJECT;

SQL> DESC SUBJECT1;

Name

SUBCODE

TITLE

SEM

CREDITS

DESC IAMARKS;

SQL> DESC IAMARKS;

Name

USN

SUBCODE

SSID

TEST1

TEST2

TEST3

FINALIA

Insertion of values to tables

INSERT INTO STUDENT VALUES ('1RN13CS020','AKSHAY','BELAGAVI',
8877881122,'M');

INSERT INTO STUDENT VALUES ('1RN13CS062','SANDHYA','BENGALURU',
7722829912,'F');

INSERT INTO STUDENT VALUES ('1RN13CS091','TEESHA','BENGALURU',
7712312312,'F');

INSERT INTO STUDENT VALUES ('1RN13CS066','SUPRIYA','MANGALURU',
8877881122,'F');

INSERT INTO STUDENT VALUES ('1RN14CS010','ABHAY','BENGALURU',
9900211201,'M');

INSERT INTO STUDENT VALUES ('1RN14CS032','BHASKAR','BENGALURU',
9923211099,'M');

INSERT INTO STUDENT VALUES ('1RN14CS025','ASMI','BENGALURU', 7894737377,'F'); INSERT INTO
STUDENT VALUES ('1RN15CS011','AJAY','TUMKUR', 9845091341,'M');

INSERT INTO STUDENT VALUES ('1RN15CS029','CHITRA','DAVANGERE',
7696772121,'F');

INSERT INTO STUDENT VALUES ('1RN15CS045','JEEVA','BELLARY', 9944850121,'M'); INSERT
INTO STUDENT VALUES ('1RN15CS091','SANTOSH','MANGALURU', 8812332201,'M');

INSERT INTO STUDENT VALUES ('1RN16CS045','ISMAIL','KALBURGI',
9900232201,'M');

INSERT INTO STUDENT VALUES ('1RN16CS088','SAMEERA','SHIMOGA',
9905542212,'F');

INSERT INTO STUDENT VALUES ('1RN16CS122','VINAYAKA','CHIKAMAGALUR',
8800880011,'M');

```
INSERT INTO SEMSEC VALUES ('CSE8A', 8,'A');
INSERT INTO SEMSEC VALUES ('CSE8B', 8,'B');
INSERT INTO SEMSEC VALUES ('CSE8C', 8,'C');
```

```
INSERT INTO SEMSEC VALUES ('CSE7A', 7,'A');
INSERT INTO SEMSEC VALUES ('CSE7B', 7,'B');
INSERT INTO SEMSEC VALUES ('CSE7C', 7,'C');
```

```
INSERT INTO SEMSEC VALUES ('CSE6A', 6,'A');
INSERT INTO SEMSEC VALUES ('CSE6B', 6,'B');
INSERT INTO SEMSEC VALUES ('CSE6C', 6,'C');
```

```
INSERT INTO SEMSEC VALUES ('CSE5A', 5,'A');
INSERT INTO SEMSEC VALUES ('CSE5B', 5,'B');
INSERT INTO SEMSEC VALUES ('CSE5C', 5,'C');
```

```
INSERT INTO SEMSEC VALUES ('CSE4A', 4,'A');
INSERT INTO SEMSEC VALUES ('CSE4B', 4,'B');
INSERT INTO SEMSEC VALUES ('CSE4C', 4,'C');
```

```
INSERT INTO SEMSEC VALUES ('CSE3A', 3,'A');
INSERT INTO SEMSEC VALUES ('CSE3B', 3,'B');
INSERT INTO SEMSEC VALUES ('CSE3C', 3,'C');
```

```
INSERT INTO SEMSEC VALUES ('CSE2A', 2,'A');
INSERT INTO SEMSEC VALUES ('CSE2B', 2,'B');
INSERT INTO SEMSEC VALUES ('CSE2C', 2,'C');
INSERT INTO SEMSEC VALUES ('CSE1A', 1,'A');
```

```
INSERT INTO SEMSEC VALUES ('CSE1B', 1,'B'); INSERT INTO
SEMSEC VALUES ('CSE1C', 1,'C');
```

```
INSERT INTO CLASS VALUES ('1RN13CS020','CSE8A');
INSERT INTO CLASS VALUES ('1RN13CS062','CSE8A');
INSERT INTO CLASS VALUES ('1RN13CS066','CSE8B');
INSERT INTO CLASS VALUES ('1RN13CS091','CSE8C');
```

```
INSERT INTO CLASS VALUES ('1RN14CS010','CSE7A');
INSERT INTO CLASS VALUES ('1RN14CS025','CSE7A');
INSERT INTO CLASS VALUES ('1RN14CS032','CSE7A');
```

```
INSERT INTO CLASS VALUES ('1RN15CS011','CSE4A');
INSERT INTO CLASS VALUES ('1RN15CS029','CSE4A');
```



```
INSERT INTO CLASS VALUES ('1RN15CS045','CSE4B');
INSERT INTO CLASS VALUES ('1RN15CS091','CSE4C');
```

```
INSERT INTO CLASS VALUES ('1RN16CS045','CSE3A');
INSERT INTO CLASS VALUES ('1RN16CS088','CSE3B');
INSERT INTO CLASS VALUES ('1RN16CS122','CSE3C');
```

```
INSERT INTO SUBJECT VALUES ('10CS81','ACA', 8, 4);
INSERT INTO SUBJECT VALUES ('10CS82','SSM', 8, 4);
INSERT INTO SUBJECT VALUES ('10CS83','NM', 8, 4);
INSERT INTO SUBJECT VALUES ('10CS84','CC', 8, 4);
INSERT INTO SUBJECT VALUES ('10CS85','PW', 8, 4);
```

```
INSERT INTO SUBJECT VALUES ('10CS71','OOAD', 7, 4);
INSERT INTO SUBJECT VALUES ('10CS72','ECS', 7, 4);
INSERT INTO SUBJECT VALUES ('10CS73','PTW', 7, 4);
INSERT INTO SUBJECT VALUES ('10CS74','DWD', 7, 4);
INSERT INTO SUBJECT VALUES ('10CS75','JAVA', 7, 4);
INSERT INTO SUBJECT VALUES ('10CS76','SAN', 7, 4);
```

```
INSERT INTO SUBJECT VALUES ('15CS51','ME', 5, 4);
INSERT INTO SUBJECT VALUES ('15CS52','CN', 5, 4);
INSERT INTO SUBJECT VALUES ('15CS53','DBMS', 5, 4);
INSERT INTO SUBJECT VALUES ('15CS54','ATC', 5, 4);
INSERT INTO SUBJECT VALUES ('15CS55','JAVA', 5, 3);
INSERT INTO SUBJECT VALUES ('15CS56','AI', 5, 3);
```

```
INSERT INTO SUBJECT VALUES ('15CS41','M4', 4, 4);
INSERT INTO SUBJECT VALUES ('15CS42','SE', 4, 4);
INSERT INTO SUBJECT VALUES ('15CS43','DAA', 4, 4);
INSERT INTO SUBJECT VALUES ('15CS44','MPMC', 4, 4);
INSERT INTO SUBJECT VALUES ('15CS45','OOC', 4, 3);
INSERT INTO SUBJECT VALUES ('15CS46','DC', 4, 3);
```

```
INSERT INTO SUBJECT VALUES ('15CS31','M3', 3, 4);
INSERT INTO SUBJECT VALUES ('15CS32','ADE', 3, 4);
INSERT INTO SUBJECT VALUES ('15CS33','DSA', 3, 4);
INSERT INTO SUBJECT VALUES ('15CS34','CO', 3, 4);
INSERT INTO SUBJECT VALUES ('15CS35','USP', 3, 3);
INSERT INTO SUBJECT VALUES ('15CS36','DMS', 3, 3);
```

```

INSERT INTO IAMARKS (USN, SUBCODE, SSID, TEST1, TEST2, TEST3) VALUES
('1RN13CS091','10CS81','CSE8C', 15, 16, 18);
INSERT INTO IAMARKS (USN, SUBCODE, SSID, TEST1, TEST2, TEST3) VALUES
('1RN13CS091','10CS82','CSE8C', 12, 19, 14);
INSERT INTO IAMARKS (USN, SUBCODE, SSID, TEST1, TEST2, TEST3) VALUES
('1RN13CS091','10CS83','CSE8C', 19, 15, 20);
INSERT INTO IAMARKS (USN, SUBCODE, SSID, TEST1, TEST2, TEST3) VALUES
('1RN13CS091','10CS84','CSE8C', 20, 16, 19);
INSERT INTO IAMARKS (USN, SUBCODE, SSID, TEST1, TEST2, TEST3) VALUES
('1RN13CS091','10CS85','CSE8C', 15, 15, 12);

```

```
SELECT * FROM STUDENT;
```

```
SQL> SELECT * FROM STUDENT1;
```

USN	SNAME	ADDRESS	PHONE	G
1RN13CS020	AKSHAY	BELAGAVI	8877881122	M
1RN13CS062	SANDHYA	BENGALURU	7722829912	F
1RN13CS091	TEESHA	BENGALURU	7712312312	F
1RN13CS066	SUPRIYA	MANGALURU	8877881122	F
1RN14CS010	ABHAY	BENGALURU	9900211201	M
1RN14CS032	BHASKAR	BENGALURU	9923211099	M
1RN15CS011	AJAY	TUMKUR	9845091341	M
1RN15CS029	CHITRA	DAVANGERE	7696772121	F
1RN15CS045	JEEVA	BELLARY	9944850121	M
1RN15CS091	SANTOSH	MANGALURU	8812332201	M
1RN16CS045	ISMAIL	KALBURGI	9900232201	M
1RN16CS088	SAMEERA	SHIMOGA	9905542212	F
1RN16CS122	VINAYAKA	CHIKAMAGALUR	8800880011	M
1RN14CS025	ASMI	BENGALURU	7894737377	F

```
SELECT * FROM SEMSEC;
```

```
SQL> SELECT * FROM SEMSEC;
```

SSID	SEM	S
CSE8A	8	A
CSE8B	8	B
CSE8C	8	C
CSE7A	7	A
CSE7B	7	B
CSE7C	7	C
CSE6A	6	A
CSE6B	6	B
CSE6C	6	C
CSE5A	5	A
CSE5B	5	B
CSE5C	5	C
CSE4A	4	A
CSE4B	4	B
CSE4C	4	C
CSE3A	3	A
CSE3B	3	B
CSE3C	3	C
CSE2A	2	A
CSE2C	2	C
CSE2B	2	B
CSE1A	1	A
CSE1B	1	B
CSE1C	1	C

```
SELECT * FROM CLASS;
SQL> SELECT * FROM CLASS;
```

USN	SSID
1RN13CS020	CSE8A
1RN13CS062	CSE8A
1RN13CS066	CSE8B
1RN13CS091	CSE8C
1RN14CS010	CSE7A
1RN14CS025	CSE7A
1RN14CS032	CSE7A
1RN15CS011	CSE4A
1RN15CS029	CSE4A
1RN15CS045	CSE4B
1RN15CS091	CSE4C
1RN16CS045	CSE3A
1RN16CS088	CSE3B
1RN16CS122	CSE3C

14 rows selected.

```
SELECT * FROM SUBJECT;
```

SUBCODE	TITLE	SEM	CREDITS
10CS81	ACA	8	4
10CS82	SSM	8	4
10CS83	NM	8	4
10CS84	CC	8	4
10CS85	PW	8	4
10CS71	OOD	7	4
10CS72	ECS	7	4
10CS73	PTW	7	4
10CS74	DWDM	7	4
10CS75	JAVA	7	4
10CS76	SAN	7	4
15CS51	ME	5	4
15CS52	CN	5	4
15CS53	DBMS	5	4
15CS54	ATC	5	4
15CS55	JAVA	5	3
15CS56	AI	5	3
15CS41	M4	4	4
15CS42	SE	4	4
15CS43	DAA	4	4
15CS44	MPMC	4	4
15CS45	OOC	4	3
15CS46	DC	4	3
15CS31	M3	3	4
15CS32	ADE	3	4
15CS33	DSA	3	4
15CS34	CO	3	4
15CS35	USP	3	3
15CS36	DMS	3	3

SELECT * FROM IAMARKS;

SQL> SELECT * FROM IAMARKS;

USN	SUBCODE	SSID	TEST1	TEST2	TEST3	FINALIA
1RN13CS091	10CS81	CSE8C	15	16	18	
1RN13CS091	10CS82	CSE8C	12	19	14	
1RN13CS091	10CS83	CSE8C	19	15	20	
1RN13CS091	10CS84	CSE8C	20	16	19	
1RN13CS091	10CS85	CSE8C	15	15	12	

Queries:

1. List all the student details studying in fourth semester 'C' section.

```
SELECT S.*, SS.SEM, SS.SEC
FROM STUDENT S, SEMSEC SS, CLASS C WHERE
S.USN = C.USN AND
SS.SSID = C.SSID AND SS.SEM = 4 AND SS.SEC='C';
```

USN	SNAME	ADDRESS	PHONE G	SEM S
1RN15CS091	SANTOSH	MANGALURU	8812332201 M	4 C

2. Compute the total number of male and female students in each semester and in each section.

```
SELECT SS.SEM, SS.SEC, S.GENDER, COUNT (S.GENDER) AS COUNT FROM
STUDENT S, SEMSEC SS, CLASS C WHERE S.USN = C.USN AND
SS.SSID = C.SSID
GROUP BY SS.SEM, SS.SEC, S.GENDER
ORDER BY SEM;
```

SEM	S	G	COUNT
3	A	M	1
3	B	F	1
3	C	M	1
4	A	F	1
4	A	M	1
4	B	M	1
4	C	M	1
7	A	F	1
7	A	M	2
8	A	F	1
8	A	M	1
8	B	F	1
8	C	F	1

3. Create a view of Test1 marks of student USN '1BI15CS101' in all subjects.

```
CREATE VIEW STU_TEST1_MARKS_VIEW AS
SELECT TEST1, SUBCODE FROM
IAMARKS
```

```
WHERE USN = '1RN13CS091';
```

TEST1 SUBCODE

15 10CS81
12 10CS82
19 10CS83
20 10CS84
15 10CS85

4. Calculate the FinalIA (average of best two test marks) and update the corresponding table for all students.

```
CREATE OR REPLACE PROCEDURE AVGMARKS IS
CURSOR C_IAMARKS IS
SELECT GREATEST(TEST1,TEST2) AS A, GREATEST(TEST1,TEST3) AS B,
GREATEST(TEST3,TEST2) AS C
FROM IAMARKS
WHERE FINALIA IS NULL
FOR UPDATE;
```

```
C_A NUMBER;
C_B NUMBER;
C_C NUMBER;
C_SM NUMBER;
C_AV NUMBER;
```

```
BEGIN
OPEN C_IAMARKS;
LOOP
    FETCH C_IAMARKS INTO C_A, C_B, C_C;
    EXIT WHEN C_IAMARKS%NOTFOUND;
    --DBMS_OUTPUT.PUT_LINE(C_A || ' ' || C_B || ' ' || C_C); IF (C_A !=
C_B) THEN
C_SM:=C_A+C_B;
    ELSE
C_SM:=C_A+C_C;
    END IF;

    C_AV:=C_SM/2;
    --DBMS_OUTPUT.PUT_LINE('SUM = '||C_SM);
    --DBMS_OUTPUT.PUT_LINE('AVERAGE = '||C_AV);
    UPDATE IAMARKS SET FINALIA=C_AV WHERE CURRENT OF C_IAMARKS;
END LOOP;
CLOSE C_IAMARKS;
END;
```

/

Note: Before execution of PL/SQL procedure, IAMARKS table contents are:

SELECT * FROM IAMARKS;

SQL> SELECT * FROM IAMARKS;

USN	SUBCODE	SSID	TEST1	TEST2	TEST3	FINALIA
1RN13CS091	10CS81	CSE8C	15	16	18	
1RN13CS091	10CS82	CSE8C	12	19	14	
1RN13CS091	10CS83	CSE8C	19	15	20	
1RN13CS091	10CS84	CSE8C	20	16	19	
1RN13CS091	10CS85	CSE8C	15	15	12	

Below SQL code is to invoke the PL/SQL stored procedure from the command line:

```
BEGIN
AVGMARKS;
END;
/
```

SQL> select * from IAMARKS;

USN	SUBCODE	SSID	TEST1	TEST2	TEST3	FINALIA
1RN13CS091	10CS81	CSE8C	15	16	18	17
1RN13CS091	10CS82	CSE8C	12	19	14	17
1RN13CS091	10CS83	CSE8C	19	15	20	20
1RN13CS091	10CS84	CSE8C	20	16	19	20
1RN13CS091	10CS85	CSE8C	15	15	12	15

5. Categorize students based on the following criterion:

If FinalIA = 17 to 20 then CAT = 'Outstanding'

If FinalIA = 12 to 16 then CAT = 'Average'

If FinalIA < 12 then CAT = 'Weak'

Give these details only for 8th semester A, B, and C section students.

```
SELECT S.USN,S.SNAME,S.ADDRESS,S.PHONE,S.GENDER, (CASE
    WHEN IA.FINALIA BETWEEN 17 AND 20 THEN 'OUTSTANDING' WHEN
    IA.FINALIA BETWEEN 12 AND 16 THEN 'AVERAGE' ELSE 'WEAK'
END) AS CAT
FROM STUDENT S, SEMSEC SS, IAMARKS IA, SUBJECT SUB
WHERE S.USN = IA.USN AND
SS.SSID = IA.SSID AND
SUB.SUBCODE = IA.SUBCODE AND
SUB.SEM = 8;
```

USN	SNAME	ADDRESS	PHONE	G	CAT
1RN13CS091	TEESHA	BENGALURU	7712312312	F	OutStanding
1RN13CS091	TEESHA	BENGALURU	7712312312	F	OutStanding
1RN13CS091	TEESHA	BENGALURU	7712312312	F	OutStanding
1RN13CS091	TEESHA	BENGALURU	7712312312	F	OutStanding
1RN13CS091	TEESHA	BENGALURU	7712312312	F	Average

PROGRAM NO 3.

Queries using aggregate functions (COUNT,AVG, MIN, MAX, SUM), Group by, Orderby.

Employee(E_id, E_name, Age, Salary)

- 1. Create Employee table containing all Records E_id, E_name, Age, Salary.**
- 2. Count number of employee names from employeetable**
- 3. Find the Maximum age from employee table.**
- 4. Find the Minimum age from employeetable.**
- 5. Find salaries of employee in Ascending Order.**
- 6. Find grouped salaries of employees.**

1. Create Employee table

```
CREATE TABLE Employee (  
    E_id INT PRIMARY  
    KEY, E_name  
    VARCHAR(50), Age INT,  
    Salary DECIMAL(10, 2)  
);
```

To insert sample records:

sql

```
INSERT INTO Employee (E_id, E_name, Age, Salary) VALUES (1,  
'Alice', 28, 50000),  
(2, 'Bob', 35, 60000),  
(3, 'Charlie', 30, 55000),  
(4, 'David', 28, 50000),  
(5, 'Eve', 40, 75000);
```

2. Count number of employee names

```
SELECT COUNT(E_name) AS Total_Employees FROM Employee;
```

3. Find the maximum age

```
SELECT MAX(Age) AS Max_Age FROM Employee;
```

4. Find the minimum age

```
SELECT MIN(Age) AS Min_Age FROM Employee;
```

5. Find salaries of employees in ascending order

```
SELECT E_name, Salary FROM  
Employee ORDER BY Salary ASC;
```

6. Find grouped salaries of employees

You can group employees by **Salary** and count how many share the same salary:

```
SELECT Salary, COUNT(*) AS Num_Employees  
FROM Employee  
GROUP BY Salary  
ORDER BY Salary;
```


PROGRAM NO 4.

Install an Open Source NoSQL Data base MongoDB & perform basic CRUD(Create, Read, Update & Delete) operations. Execute MongoDB basic Queries using CRUD

Step 1: Installation Guide (Ubuntu Linux / Windows)

- **Ubuntu:** Run `sudo apt-get install mongodb`
- **Start Service:** `sudo systemctl start mongodb`
- **Launch Shell:** Type `mongosh` or `mongo` in the terminal.

Step 2: Execution of CRUD Operations

1. Create (Insert Documents)

JavaScript

```
// Switch to a new database
use vtu_lab;
```

```
// Insert a single document
```

```
db.students.insertOne({
  usn: "1BI15CS101",
  name: "Rahul Kumar",
  sem: 4,
  branch: "CSE"
});
```

```
// Insert multiple documents
```

```
db.students.insertMany([
  { usn: "1BI15CS102", name: "Ananya", sem: 4, branch: "CSE" },
  { usn: "1BI15CS103", name: "Vijay", sem: 6, branch: "ISE" }
]);
```

2. Read (Query Documents)

JavaScript

```
// Retrieve all records
db.students.find().pretty();
```

```
// Filter documents by criteria
```

```
db.students.find({ branch: "CSE" });
```

3. Update (Modify Documents)

JavaScript

```
// Update single document field
```

```
db.students.updateOne(
  { usn: "1BI15CS101" },
  { $set: { sem: 5 } }
);
```

4. Delete (Remove Documents)

JavaScript

```
// Delete a specific record
db.students.deleteOne({ usn: "1BI15CS103" });
```

1. Verifying "Create" (Insert)

After running the `insertOne` or `insertMany` commands, MongoDB returns an object confirming the operation.

- **Command:** `db.students.insertOne({ usn: "1BI15CS101", name: "Rahul Kumar" })`

- **Expected Output:**

```
JSON
{
  "acknowledged": true,
  "insertedId": ObjectId("6695a1b2c3d4e5f6g7h8i9j0")
}
```

2. Verifying "Read" (Query)

This is how you view the data you just stored.

- **Command:** `db.students.find({ branch: "CSE" }).pretty()`
- **Expected Output:**

```
JSON
[
  {
    "_id": ObjectId("6695a1b2c3d4e5f6g7h8i9j0"),
    "usn": "1BI15CS101",
    "name": "Rahul Kumar",
    "sem": 4,
    "branch": "CSE"
  }
]
```

3. Verifying "Update"

To check if the update worked, perform a find query on that specific record.

- **Command:** `db.students.findOne({ usn: "1BI15CS101" })`
- **Expected Output:** You will see the "sem": 5 field, reflecting the change you made in the update command.

4. Verifying "Delete"

- **Command:** `db.students.deleteOne({ usn: "1BI15CS103" })`
- **Expected Output:**

```
JSON
{
  "acknowledged": true,
  "deletedCount": 1
}
```

